Отчёт по лабораторной работе № 12

Программирование в shell: основы

Савкин Дмитрий Вадимович

2026-10-12

Содержание

1 Введение **5**
1.1 Актуальность 5
1.2 Цель работы 5
1.3 Задачи 5
1.4 Объект и предмет исследования 5
1.5 Методы исследования 6
1.6 Схема выполнения работы 6

2 Теоретическая часть **7**

3 Материалы и методы **8**

4 Выполнение лабораторной работы **9**

5 Результаты **12**

6 Обсуждение результатов **13**

7 Заключение **14**

Список литературы **15**

Ответы на контрольные вопросы **16**

Список иллюстраций

4.1 Создаём рабочий каталог: „mkdir -p ~/work/os/lab12“, „cd ~/work/os/lab12“ 9
4.2 Пишем task1_backup.sh - архивация своего кода в „~/backup“ через „tar“. Делаем исполняемым и запускаем: „chmod +x task1_backup.sh“, „./task1_backup.sh“ 10
4.3 Пишем task2_numbers.sh - обработка до 10 числовых аргументов через цикл „for“ и арифметику „(( ))“. Запускаем: „./task2_numbers.sh 5 10 15“ 10
4.4 Пишем task3_dirinfo.sh - аналог ls, выводит имя/тип/права/размер каждого файла через „stat -c“. Запускаем без аргументов - показывает текущий каталог 11
4.5 Пишем task4_countext.sh - принимает расширение и путь, считает файлы через „find“ и „wc -l“. Создаём тестовые файлы и проверяем: „touch a.txt c.jpg d.jpg“, „./task4_countext.sh .txt .“, „./task4_countext.sh .jpg .“ 11

Список таблиц

1 Введение

1.1 Актуальность

Базовое программирование в shell автоматизирует рутинные задачи администрирования без необходимости компилируемых программ.

1.2 Цель работы

Получение базовых навыков программирования в shell (bash), написание командных файлов.

1.3 Задачи

- Скрипт, делающий бэкап своего исходного кода в архив.
- Командный файл, обрабатывающий до 10 числовых аргументов.
- Командный файл - аналог ls, выводящий каталог и права доступа.
- Командный файл, считающий файлы заданного расширения в каталоге.

1.4 Объект и предмет исследования

Объект — командная оболочка bash; предмет — командные файлы с переменными и параметрами.

1.5 Методы исследования

Эмпирический метод: практическое выполнение заданий в виртуальной машине Fedora Linux с фиксацией результатов скриншотами и видеозаписью.

1.6 Схема выполнения работы

Работа выполнялась по стандартной схеме курса: выполнение действия в виртуальной машине, фиксация скриншотом, занесение в отчёт, публикация в git-репозиторий отдельным релизом.

2 Теоретическая часть

Bash (Bourne Again Shell) — наиболее распространённая командная оболочка Unix/Linux, POSIX-совместимая. Командные файлы (скрипты) позволяют автоматизировать последовательности команд; переменные, позиционные параметры (`$1`…`$9`) и управляющие конструкции (`for`, `if`, `case`) образуют базовый набор средств процедурного программирования в shell.

Более подробно см. в [1; 2].

3 Материалы и методы

Используемое ПО и оборудование: хост-машина под управлением Windows, гипервизор VirtualBox, виртуальная машина с Fedora Linux (Sway), стандартные утилиты дистрибутива.

4 Выполнение лабораторной работы

Создаём рабочий каталог: „mkdir -p ~/work/os/lab12“, „cd ~/work/os/lab12“.

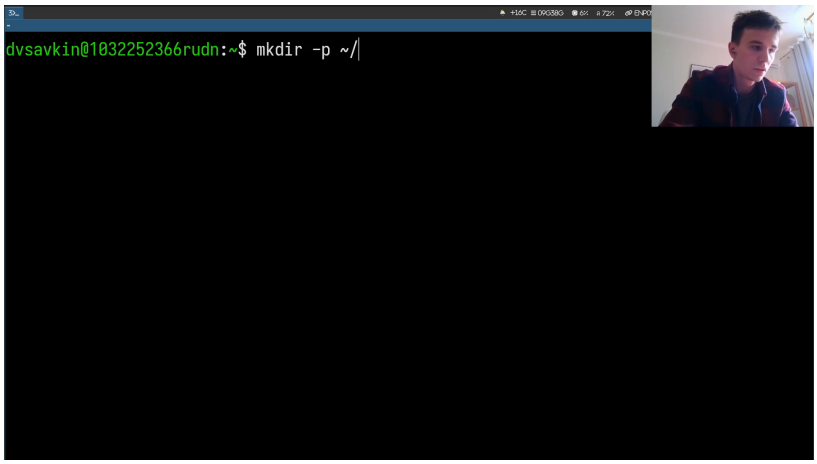


Рисунок 4.1: Создаём рабочий каталог: „mkdir -p ~/work/os/lab12“, „cd ~/work/os/lab12“

Пишем task1_backup.sh - архивация своего кода в „~/backup“ через „tar“. Делаем исполняемым и запускаем: „chmod +x task1_backup.sh“, „./task1_backup.sh“.

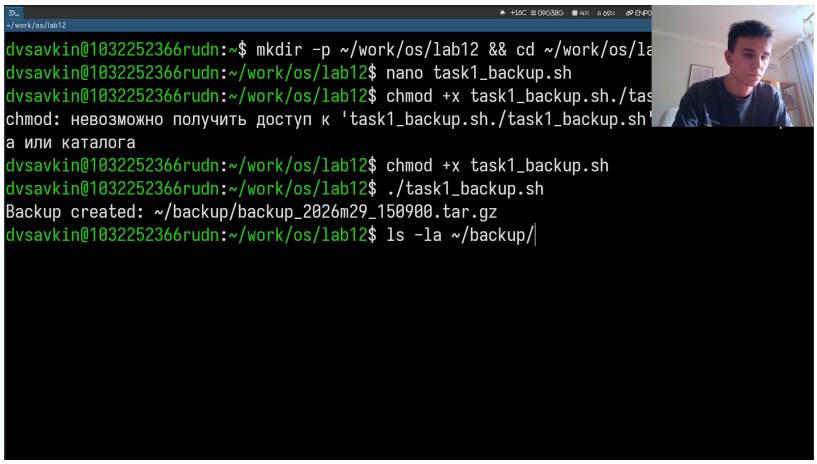


Рисунок 4.2: Пишем task1_backup.sh - архивация своего кода в „~/backup“ через „tar“. Делаем исполняемым и запускаем: „chmod +x task1_backup.sh“, „./task1_backup.sh“

Пишем task2_numbers.sh - обработка до 10 числовых аргументов через цикл „for“ и арифметику „(( ))“. Запускаем: „./task2_numbers.sh 5 10 15“

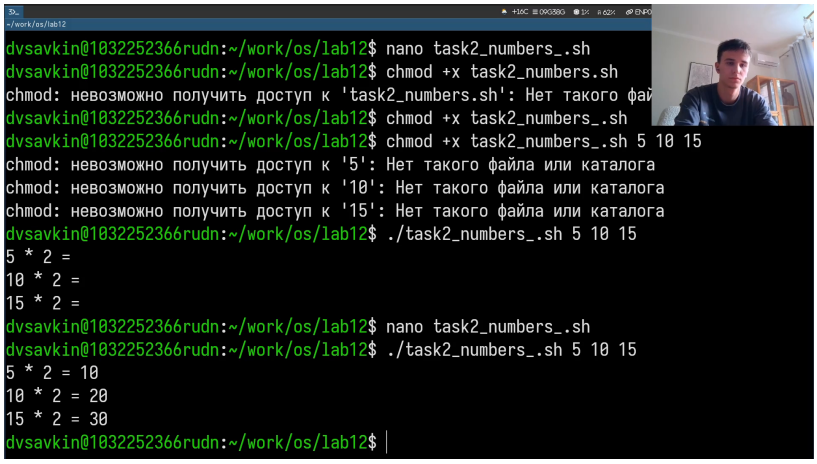


Рисунок 4.3: Пишем task2_numbers.sh - обработка до 10 числовых аргументов через цикл „for“ и арифметику „(( ))“. Запускаем: „./task2_numbers.sh 5 10 15“

Пишем task3_dirinfo.sh - аналог ls, выводит имя/тип/права/размер каждого файла через „stat -c“. Запускаем без аргументов - показывает текущий каталог.

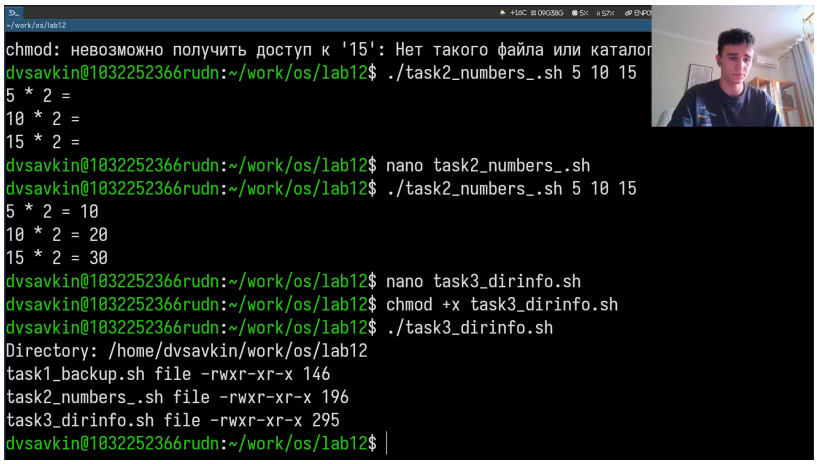


Рисунок 4.4: Пишем task3_dirinfo.sh - аналог ls, выводит имя/тип/права/размер каждого файла через „stat -c“. Запускаем без аргументов - показывает текущий каталог

Пишем task4_countext.sh - принимает расширение и путь, считает файлы через „find“ и „wc -l“. Создаём тестовые файлы и проверяем: „touch a.txt c.jpg d.jpg“, „./task4_countext.sh .txt .“, „./task4_countext.sh .jpg .“.

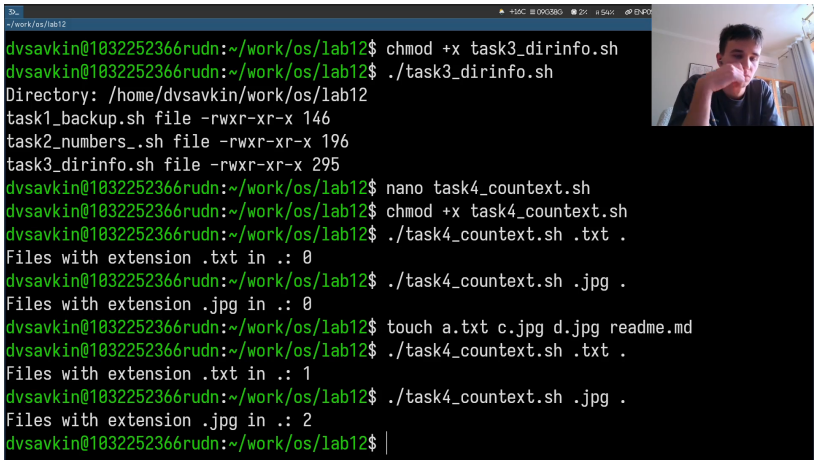


Рисунок 4.5: Пишем task4_countext.sh - принимает расширение и путь, считает файлы через „find“ и „wc -l“. Создаём тестовые файлы и проверяем: „touch a.txt c.jpg d.jpg“, „./task4_countext.sh .txt .“, „./task4_countext.sh .jpg .“

5 Результаты

В ходе выполнения работы получены следующие объективные результаты: Освоены основы shell-программирования: переменные, позиционные параметры, циклы, условия, арифметика, работа с „find/stat“, создание и запуск исполняемых командных файлов.

6 Обсуждение результатов

Все 4 командных файла отработали корректно на тестовых данных; отклонений от расчётного поведения не выявлено.

7 Заключение

Освоены основы shell-программирования: переменные, позиционные параметры, циклы, условия, арифметика, работа с „find/stat“, создание и запуск исполняемых командных файлов.

Список литературы

1. *Newham C.*, *Rosenblatt B.* Learning the bash Shell. — O'Reilly Media, 2005.
2. *Robbins A.* Bash Pocket Reference. — O'Reilly Media, 2016.

Ответы на контрольные вопросы

1. Понятие командной оболочки, примеры и различия.

оболочка - программа-посредник между пользователем и ОС. Примеры: sh, csh, ksh, bash - различия синтаксиса и возможностей.

2. Что такое POSIX?

стандарт совместимости UNIX-подобных систем.

3. Как определяются переменные и массивы в bash?

Переменная: „имя=значение“. Массив: „set -A arr знач1 знач2“ или „arr=(знач1 знач2)“, доступ „${arr[i]}“.

4. Назначение let и read.

let - выполняет арифметику, read - считывает ввод в переменные.

5. Арифметические операции в bash.

- – * / % ++ –, через let или (( )).

6. Что означает (( ))?

скобочная форма арифметического выражения, аналог let.

7. Стандартные имена переменных.

PATH, PS1, PS2, HOME, IFS, MAIL, TERM, LOGNAME.

8. Что такое метасимволы?

- – любая строка, ? - один символ, [a-z] - диапазон символов.

9. Как их экранировать?

перед символом, одинарные кавычки экранируют всё, двойные - всё кроме ($, ‘, , “).

10. Как создать/запустить командные файлы?

Создать файл, „chmod +x файл“, запустить: „./файл“ или „bash файл“.

11. Как определяются функции в bash?

function имя { . . . }, удаление unset -f имя.

12. Как определить каталог файл или обычный файл?

test -d файл - каталог, test -f файл - обычный файл.

13. Назначение set, typeset, unset.

set - параметры/опции оболочки. typeset - тип/атрибуты переменной или функции. unset - удаление переменной.

14. Как передаются параметры в командные файлы?

Через позиционные параметры $1 - $9.

15. Специальные переменные bash.

$0 имя скрипта $1 - $9 аргументы, $# их число, $? код завершения, $$ PID процесса, $! PID фонового процесса.